



Secure Password Manager (Java)

Programming in Java – BYOP Project Report (Vityarthi)

Name- Nisha

Reg. Number- 24BAI10441

Slot- F11+F12

Faculty- Dr. Ganesan G

Date of Submission- 31st March 2026

Abstract

This project implements a **Secure Password Manager** using Java as a console-based application. It helps users store and manage their credentials securely, addressing the common problem of unsafe password storage. The system provides features such as master password authentication, adding, viewing, and deleting passwords. Basic security is ensured using Base64 encoding, and data is stored using file handling for persistence. The project demonstrates the use of Object-Oriented Programming and core Java concepts to solve a real-world problem. It can be further enhanced with advanced encryption and a graphical user interface.

1. Introduction

In today's digital world, users manage multiple accounts across various platforms, each requiring secure authentication. As a result, remembering numerous passwords becomes difficult, leading many users to adopt unsafe practices such as using weak passwords, repeating the same password across services, or storing them in unsecured formats. These practices increase the risk of data breaches and unauthorized access.

The Secure Password Manager is designed to address this problem by providing a simple and efficient way to store and manage user credentials securely. This project is implemented as a console-based application using Java and focuses on applying core programming concepts such as Object-Oriented Programming, file handling, and modular design.

The system allows users to authenticate using a master password and perform operations such as adding, viewing, and deleting stored credentials. Basic encoding techniques are used to enhance security, while data persistence is achieved through file storage.

This project demonstrates how fundamental Java concepts can be used to develop a practical solution to a real-world problem, with scope for future enhancements such as advanced encryption methods and graphical user interfaces.

2. Problem Statement

Nowadays, almost everyone uses multiple apps and websites that require passwords. Remembering all of them can be difficult, so people often end up using the same password everywhere or writing them down in notes or files, which is not safe. This increases the chances of accounts getting hacked or sensitive information being exposed.

Because of this, there is a need for a simple and secure way to store and manage passwords. The system should be easy to use and should help users keep their credentials organized without compromising security.

This project focuses on creating a Secure Password Manager using Java that allows users to safely store, access, and manage their passwords in a better and more organized way.

3. Objectives

- To create a simple application that helps users store their passwords in one place instead of remembering multiple credentials.

- To provide basic security by using a master password so that only the user can access the stored data.
 - To allow users to easily add, view, and delete their saved passwords whenever needed.
 - To use file handling so that the data remains saved even after the program is closed.
 - To apply core Java concepts like Object-Oriented Programming and modular coding in a practical project.
 - To develop a system that is easy to use and solves a real-life problem faced by many users.
-

4. System Features

- **Master Password Authentication**
The system requires a master password to access the application, ensuring that only the authorized user can view or manage stored credentials.
 - **Add New Credentials**
Users can store new login details by entering the website name, username, and password.
 - **View Stored Passwords**
The application allows users to view all saved credentials in a clear and organized format.
 - **Delete Credentials**
Users can remove any saved password if it is no longer needed.
 - **Basic Security (Encoding)**
Passwords are encoded using Base64 before being stored, so they are not directly visible in plain text.
 - **File-Based Storage**
All data is stored in a local file, ensuring that passwords remain saved even after the program is closed.
 - **Simple Console Interface**
The system uses a menu-driven console interface, making it easy to use without any complex setup.
-

5. Methodology / Approach

The project was developed step by step by first understanding the problem of managing multiple passwords and identifying the basic features required in a password manager. A simple console-based approach was chosen to keep the system easy to build and use.

The application is designed using Object-Oriented Programming concepts, where different tasks are divided into separate classes. The main class controls the program flow, while other classes handle operations like storing passwords and managing file data. This modular approach makes the code easier to understand and maintain.

For storing data, file handling is used so that all passwords are saved in a text file and can be accessed even after the program is closed. To provide basic security, passwords are encoded using Base64 before being stored.

The system follows a menu-driven approach where users can select options such as adding, viewing, or deleting credentials. Each function is implemented separately and connected through the main program, ensuring smooth execution.

Overall, the approach focuses on simplicity, proper organization of code, and solving a real-world problem using core Java concepts.

6. Key Design Decisions

- A console-based approach was chosen to keep the application simple and easy to use.
 - Object-Oriented Programming was used to organize the code into different classes.
 - File handling was used instead of a database to keep the project lightweight.
 - Base64 encoding was applied to provide basic security for stored passwords.
 - A master password was added to restrict unauthorized access.
 - A menu-driven design was used for easy navigation.
-

7. Technologies Used

- Java – Used as the main programming language to develop the application.
 - Object-Oriented Programming (OOP) – Used to organize the code into classes and make it modular.
 - File Handling – Used to store and retrieve password data from a text file.
 - Scanner Class – Used to take user input from the console.
 - Base64 Encoding – Used to provide basic security for stored passwords.
-

8. Detailed Code Explanation

The following section presents the complete implementation of the Library Management System along with detailed explanations of each part of the code to provide a clear understanding of how the system works.

8.1 Import Statements

```
import java.util.*;
import java.io.*;
import java.util.Base64;
```

These import statements are used to include necessary classes for the program. The java.util package provides utility classes like Scanner and ArrayList, java.io is used for file handling operations, and Base64 is used for encoding and decoding passwords.

8.2 Main Class

```
public class Main {
```

This is the main class of the application. It contains the main() method, which acts as the entry point of the program.

8.3 Main Method

```
public static void main(String[] args) {
```

The main method controls the overall execution of the program. It initializes objects, handles user input, and displays the menu-driven interface.

8.4 Object Initialization

```
Scanner sc = new Scanner(System.in);
PasswordService service = new PasswordService();
```

- Scanner object is used to take input from the user.
- PasswordService object is created to access methods related to password operations.

8.5 Master Password Authentication

```
System.out.print("Enter Master Password: ");
String master = sc.nextLine();
```

```
if (!master.equals("admin123")) {
    System.out.println("Access Denied!");
}
```

```
    return;  
}
```

This section ensures security by asking the user to enter a master password. If the password is incorrect, the program terminates, preventing unauthorized access.

8.6 Menu-Driven System

```
int choice;  
  
do {  
    System.out.println("\n1. Add Password");  
    System.out.println("2. View Passwords");  
    System.out.println("3. Delete Password");  
    System.out.println("4. Exit");  
  
    System.out.print("Enter choice: ");  
    choice = sc.nextInt();  
    sc.nextLine();  
}
```

This block displays menu options and takes user input. The loop continues until the user selects the exit option.

8.7 Switch Case Execution

```
switch (choice) {  
    case 1: service.addPassword(); break;  
    case 2: service.viewPasswords(); break;  
    case 3: service.deletePassword(); break;  
    case 4: System.out.println("Exiting..."); break;  
    default: System.out.println("Invalid choice!");  
}
```

The switch-case structure executes different operations based on the user's choice, ensuring proper navigation within the application.

8.8 PasswordService Class

```
public class PasswordService {
```

This class handles all core functionalities such as adding, viewing, and deleting passwords.

8.9 Add Password Function

```
public void addPassword() {
```

This function collects user input (website, username, and password), encodes the password using Base64, and stores it in a file through the FileHandler class.

8.10 View Password Function

```
public void viewPasswords() {
```

This function reads all stored data from the file, decodes the passwords, and displays them in a readable format.

8.11 Delete Password Function

```
public void deletePassword() {
```

This function allows the user to delete a specific password entry based on the website name and updates the file accordingly.

8.12 Base64 Encoding

```
String encodedPass = Base64.getEncoder().encodeToString(pass.getBytes());
```

This line encodes the password before storing it, ensuring that it is not saved in plain text.

8.13 FileHandler Class

```
public class FileHandler {
```

This class is responsible for handling file operations such as reading, writing, and updating stored data.

8.14 File Path Declaration

```
private final String FILE_NAME = "data/passwords.txt";
```

This variable defines the file path where all password data is stored.

8.15 Write to File Function

```
public void writeFile(String data) {
```

This function writes new password data into the file using append mode, ensuring that existing data is not overwritten.

8.16 Read from File Function

```
public List<String> readFile() {
```

This function reads all stored records from the file and returns them as a list for processing.

8.17 Overwrite File Function

```
public void overwriteFile(List<String> data) {
```

This function updates the file by rewriting all records, mainly used when deleting a password entry.

9. GitHub Repository

The complete project is available at:

 <https://github.com/nisha-181206/secure-password-manager>

The repository includes the source code, README file explaining setup and usage, and the data file, ensuring transparency and reproducibility.

10. Results/Output

The Secure Password Manager was successfully implemented and tested. The application allows users to securely add, view, and delete passwords through a simple menu-driven interface. It uses a master password for authentication and stores data in a file, ensuring persistence. All functionalities worked correctly, and the system effectively manages user credentials in a secure and organized manner.


```
Enter Master Password: admin123
Login Successful!

===== PASSWORD MANAGER =====
1. Add Password
2. View Passwords
3. Delete Password
4. Exit
Enter choice: 1
Enter Website: github.com
Enter Username: nisha_181206
Enter Password: Projects@2026
? Password Saved!

===== PASSWORD MANAGER =====
1. Add Password
2. View Passwords
3. Delete Password
4. Exit
Enter choice: 1
Enter Website: gmail.com
Enter Username: Nisha
Enter Password: Engineer@2024
? Password Saved!

===== PASSWORD MANAGER =====
1. Add Password
2. View Passwords
3. Delete Password
4. Exit
Enter choice: 2

--- Saved Passwords ---
Website: github.com | Username: nisha_181206 | Password: Projects@2026
Website: gmail.com | Username: Nisha | Password: Engineer@2024
Website: github.com | Username: nisha_181206 | Password: Projects@2026
Website: gmail.com | Username: Nisha | Password: Engineer@2024
```

```
data > read.csv("passwords.txt")
  1  github.com,nisha_181206,UHJvamVjdHNAMEjAeNg==
  2  gmail.com,Nisha,RW5naW5lZXXAMEjAeNA==
```

11. Challenges Faced

During the development of this project, a few challenges were encountered. One of the main difficulties was handling file operations properly, especially while updating and deleting specific records without losing existing data. Managing user input in a menu-driven program also required careful handling to avoid errors.

Another challenge was implementing basic security for storing passwords. Understanding and applying encoding techniques like Base64 took some effort. Organizing the code into

different classes and ensuring smooth interaction between them was also slightly challenging at the beginning.

However, these challenges were resolved through practice and testing, which helped improve understanding of core Java concepts.

12. Limitations

The current system has some limitations. It uses basic Base64 encoding, which is not a strong security method and can be easily decoded. The application is console-based, so it lacks a user-friendly graphical interface.

It also stores data in a simple text file instead of a database, which may not be suitable for handling large amounts of data. Additionally, the system does not include features like password updating, searching, or strong password validation.

These limitations can be improved in future versions of the project.

13. Depth of Reflection

This project helped me understand how basic programming concepts can be applied to solve real-life problems. While developing the Secure Password Manager, I learned how to use Object-Oriented Programming, file handling, and modular design in a practical way.

I also realized the importance of writing clean and organized code, as it makes the program easier to manage and improve. Working on this project improved my problem-solving skills, especially while handling errors and managing data efficiently.

Overall, this project gave me confidence in building simple applications independently and helped me gain a better understanding of how real-world software systems work.

14. Learning Outcomes

Through this project, I learned how to apply core Java concepts like Object-Oriented Programming, file handling, and modular coding in a real application. I also understood how to manage data using files and how basic security techniques like encoding can be implemented. This project improved my logical thinking, debugging skills, and overall confidence in building small software systems independently.

15. Conclusion

The Secure Password Manager project was successfully developed to provide a simple and organized way to store and manage passwords. It meets the main objective of improving password management using basic security and file handling techniques. The project demonstrates how Java can be used to solve real-world problems effectively in a simple and structured manner.

16. Future Scope

The project can be improved by adding advanced security features like encryption (AES), a graphical user interface for better usability, and database integration for handling larger data. Additional features such as password update, search functionality, and password strength checking can also be included to make the system more efficient and user-friendly.